

T07 과제 체크리스트 — 플랜두씨 다이어리 2: 인증 붙이기

전제: T06(계획·실행·돌아보기 앱, 이 저장소)에 이어서 인증(가입/로그인/로그아웃)을 붙이는 과제. 계획시간 6~8h, 실제 사용 5일(서로 다른 실제 날짜). 채점자는 **제출물을 읽기만** 하고 내 계정으로 로그인하지 않으므로, "막았다"는 문장이 아니라 ****막는 장면(요청+응답)****을 제출해야 통과.

2026-09-02 기준: 카드 1~4(인증 구현 자체)는 실제 배포 API에 요청을 보내 성공/거절 응답을 직접 캡처해 검증 완료. 원본 기록: `docs/t07_evidence_raw.json`, 정리된 설명서: `T07_AUTH_EXPLANATION.md`. 시크릿 창 접근(C01)도 사용자가 직접 확인 완료. 카드 5의 5일 실사용만 **의도적으로 1일치만 채우고 제출**하기로 결정(지어낸 기록 대신 미완을 그대로 밝힘).

카드 1 — 무엇으로 붙일지 고르고, 이유를 적기

첫 행동

- ✓ 직접구현(bcrypt+JWT)을 고르고 이름·버전을 적어둠 — `T07_AUTH_EXPLANATION.md` ①
- ✓ 함께 검토한 Supabase Auth와 고르지 않은 이유 한 문장 — `T07_AUTH_EXPLANATION.md` ②
- ✓ 로그인하지 않은 채 자료 화면을 열면 로그인 화면만 보임 (프론트 `js/main.js` 의 `checkSession()`)
- ✓ T06 실테이터를 내 계정(`test1@test.com`)으로 이관 — `sql/migrate-t06-data-to-owner.sql` 실행 완료

통과 기준

- ✓ **T07-C91** 직접구현(bcrypt+JWT)임이 적혀 있다.
- ✓ **T07-C92** `bcryptjs@2.4.3`, `jsonwebtoken@9.0.2`, `cookie@0.6.0` 버전 명시.
- ✓ **T07-C93** Supabase Auth를 검토했으나 고르지 않은 이유 명시.
- ✓ **T07-C94** 가입 성공 — 실제 요청 `POST /api/auth/signup` → `201`.
- ✓ **T07-C95** 로그인 성공 — `POST /api/auth/login` → `200`.
- ✓ **T07-C96** 로그아웃 성공 — `POST /api/auth/logout` → `200 {"ok":true}`.
- ✓ **T07-C97** 비로그인 자료 요청 → `401`, 프론트도 로그인 화면만 노출.
- ✓ **T07-C98** 같은 이메일 재가입 → `409` 이미 가입된 이메일입니다.
- ✓ **T07-C99** 틀린 비번/미존재 계정 응답 메시지 동일함을 직접 비교 확인(`identical: true`).
- ✓ **T07-C100** T06 데이터 이관 완료(`test1@test.com`).

남길 것

- ✓ 고른 방식·이름·버전, 고르지 않은 방법과 이유 — `T07_AUTH_EXPLANATION.md`
- ✓ 가입·로그인·로그아웃 요청/응답 — `docs/t07_evidence_raw.json`
- ✓ 비로그인 시 로그인 화면 노출 — 코드/동작으로 확인됨(스크린샷은 제출 시 추가 캡처 권장)

카드 2 — 비밀번호를 어떻게 맡아 두는지 보이기

통과 기준

- ☒ **T07-C101** bcrypt(`bcryptjs`) 명시.
- ☒ **T07-C102** 고른 이유(널리 쓰는 라이브러리에 맡겨 검증 비용을 줄임) 명시.
- ☒ **T07-C103** 저장된 해시 값 확인 — `docs/screenshots/t07_users_table_password_hash.png` (모든 행이 `$2a$12$...` 형태, 입력한 비밀번호 글자 그대로 보이지 않음).
- ☒ **T07-C104** 같은 비밀번호(`Password!123`)로 만든 alice/bob 여러 테스트 계정의 `password_hash` 값이 스크린샷에서 서로 다름을 확인(bcrypt는 계정마다 다른 salt를 자동으로 붙이므로).
- ☒ **T07-C105** 로그인 요청/응답 기록에 비밀번호 원문 없음 확인.
- ☐ **T07-C106** 서버 로그(Vercel Functions 로그)까지는 AI가 접근 권한이 없어 미확인 — 사용자가 Vercel 대시보드에서 최종 확인 권장.
- ☒ **T07-C107** `bcryptjs`에 맡겼음을 명시(직접 해시 알고리즘 구현 안 함).

남길 것

- ☒ 고른 방법의 이름과 이유
 - ☒ 저장된 비밀번호 값(입력 글자 안 보임) — `docs/screenshots/t07_users_table_password_hash.png`
 - ☒ 같은 비밀번호로 만든 두 계정의 저장값이 다른 결과 — 위 스크린샷에서 확인
 - ☒ 로그인 요청·응답 기록에 원문 없는 결과
-

카드 3 — 들어온 사람을 어떻게 기억하는지 보이기

통과 기준

- ☒ **T07-C108** 토큰(JWT) + 서버 측 세션 레코드 방식 명시.
- ☒ **T07-C109** 로그아웃 전 성공(200)·후 같은 값 재요청 거절(401) 나란히 기록.
- ☒ **T07-C110** 같은 주소(`/api/data/plans`)·같은 방식(GET), 로그아웃 여부만 다름 확인.
- ☒ **T07-C111** 만료 7일, `httpOnly` 쿠키로 명시(`api/_lib/auth.js` 의 `SESSION_DAYS`).
- ☒ **T07-C112** 로그인 응답 바디에 토큰 없음(`{"email":...}` 만 있음) 확인.
- ☒ **T07-C113** 저장소 전체 스캔 결과 `JWT_SECRET` 실제 값 없음(env var 이름만 참조).
- ☒ **T07-C114** 로그아웃 후 이전 세션 값 거절 확인(비밀번호 변경 기능은 미구현이라 ⑥에 기재).
- ☒ **T07-C115** 기록에 세션 쿠키 값을 `mask()` 로 가려서 저장.

남길 것

- ☒ 무엇으로 사람을 알아보는지 적은 문장
 - ☒ 로그인 상태 성공 + 로그아웃 뒤 거절 응답
 - ☒ 만료 시각과 끊기는 시간
 - ☒ 비밀키가 어디에도 없는 결과
-

카드 4 — 남의 자료가 안 열리는 것을 보이기

통과 기준

- ☒ **T07-C116** Alice/Bob 두 계정 생성 + 각자 계획 추가(비밀번호 미기재).
- ☒ **T07-C117** Bob→Alice 읽기(수정이력, 할일목록) 요청·거절 응답 기록.
- ☒ **T07-C118** Bob→Alice 계획 수정 요청·거절(404) 기록.
- ☒ **T07-C119** Bob→Alice 할 일 삭제 요청·거절(404) 기록.
- ☒ **T07-C120** 반대 방향(Alice→Bob) 읽기·수정·삭제 전부 동일하게 거절 기록.
- ☒ **T07-C121** 전부 404(존재 자체를 감춤)로 통일.
- ☒ **T07-C122** 공격 시도 전후 상대방 자료 내용·존재 여부 그대로임을 확인.
- ☒ **T07-C123** 요청 본문에 다른 계정 id를 끼워 넣어도 무시되고 실제 세션 소유자로 생성됨 확인.
- ☒ **T07-C124** 비로그인 직접 요청 → 401 거절 기록.
- ☒ **T07-C125** 목록 응답에 상대방 자료 0건 확인.
- ☒ **T07-C126** 거절을 만드는 소스 위치(`api/data/*.js` 의 `.eq('user_id', userId)`) 명시.

남길 것

- ☒ 계정 두 개 자료 넣은 기록(비밀번호 없이)
- ☒ 양방향 읽기·수정·삭제 요청과 거절 응답
- ☒ 주소·본문에 남의 계정 적어 보낸 요청과 응답
- ☒ 로그인하지 않은 직접 요청과 거절 응답
- ☒ 목록 응답에 남의 자료 없는 결과
- ☒ 거절 전후 반대편 자료 그대로인 결과 + 소스 위치

카드 5 — 설명서로 묶고, 잠금 앱으로 5일 써보기

인증 구현 설명서 (여섯 항목)

- ☒ **T07-C127** 여섯 항목 각각 나누어 작성 — `T07_AUTH_EXPLANATION.md`
- ☒ **T07-C128** 가입·로그인·로그아웃·자료조회 네 흐름의 소스 위치 표로 명시.
- ☒ **T07-C129** 다섯 가지 확인(비밀번호 저장/세션 만료·로그아웃/비로그인 차단/계정 간 읽기·수정·삭제/목록·스푸핑) 성공·거절 나란히 기재.
- ☒ **T07-C130** 미구현 4가지(무차별 대입 방어, 비밀번호 재설정, 이메일 인증, 로그인 로그)와 위험성 기재.
- ☒ **T07-C131** 저장소 전체 스캔으로 원문 비밀값 없음 확인.

5일 실사용 기록 — ⚠️ 의도적으로 1일치만 채우고 제출하기로 결정함

이 항목은 "지어낸 날짜가 아니라 실제로 다른 날짜에 걸쳐 쓴 기록"을 요구한다. 5일을 채우려면 실제로 5일이 걸리므로, 이번 제출에서는 **1일차(2026-09-02)만 진짜로 기록하고 나머지 4일은 채우지 못했음을 그대로 밝히기로 했다**(지어낸 기록으로 채우지 않음).

- ☒ **T07-C04** 1일차 질문: "오늘 계획한 할 일을 얼마나 완료했는가?"
- ☒ **T07-C05** 관찰 지표: 완료한 할 일 개수
- ☒ **T07-C06** 단위: 건(개)
- ☐ **T07-C07** 서로 다른 실제 날짜 5일 — **1일(2026-09-02)만 있음, 4일 부족**
- ☒ **T07-C08** 1일차 기록에 계산 규칙(그날 상태가 완료로 바뀐 할 일 수를 그대로 셈) 적용 — docs/screenshots/t07_day1_review.png (완료 2건), t07_day1_execution_log.png (실행 기록 추가)
- ☐ **T07-C23~C27** 결측/중복/이상치/반올림/주 시작 요일 처리 규칙 — 5일 데이터가 없어 미작성
- ☐ **T07-C09~C15** 3일차 앞 계획 규칙 변경 기록 및 전후 비교 — **미완(2·3일차 자체가 없음)**
- ☐ **T07-C132** 화면 합계·평균과 손 계산 대조 — 1일치 기준으로는 대조 가능(완료 2건 직접 셈과 일치), 5일 비교는 미완

그 외 마무리

- ☒ **T07-C133** 내보내기 API(/api/data/export) 구현 완료(T06 때 만든 파일 다운로드 로직 재사용).
- ☒ **T07-C134** 계정 삭제 시 FK cascade로 전 자료 삭제(api/data/account.js).
- ☒ **T07-C03** 결과물 첫 화면이 로그인 화면이고 계정 없이 열림, 로그인 뒤 내 기록만 비공개로 보임.
- ☒ **T07-C39** 짧은 확인 방법 4줄 — README.md 의 "짧은 확인 방법 (T07, 4줄)"
- ☒ **T07-C40** AI/직접판단/AI 미채택 — T07_AUTH_EXPLANATION.md ⑤
- ☒ **T07-C46** 저장소 전체 스캔으로 비밀값 원문 없음 확인.
- ☒ **T07-C01** 시크릿 창 접근 최종 확인 — 사용자가 직접 시크릿 창에서 배포 URL을 열어 로그인 화면이 계정 없이 정상적으로 뜨는 것을 확인함.
- ☒ **T07-C77** T07이 이어 쓴 T06 결과(t06-final 태그, 커밋 43097cc) 명시.
- ☒ **T07-C78** t06-final 태그가 현재 main 브랜치의 조상 커밋으로 포함됨(연속 커밋).

남길 것

- ☒ 여섯 항목으로 나눈 인증 구현 설명서
- ☒ 확인 다섯 가지의 성공/거절 요청
- ☐ 서로 다른 날짜 5일의 기록 + 규칙 변경 — **진행 예정**
- ☐ 화면 합계·평균과 손 계산 대조 — 5일 데이터 이후
- ☒ 내보낸 파일 API + 계정 삭제 안내
- ☒ 비밀번호·토큰·비밀키가 모두 가려진 제출물

남은 할 일 (제출 전)

1. 카드 5와 5일 실사용 — **1일치만 채우고 제출하기로 결정**(T07-C07/C09~C27/C132 일부 미충족을 그대로 인정)
2. 시크릿 창 접근 확인 — 완료 (로그인 화면 정상 노출 확인됨)
3. (선택) test1@test.com 에 섞여 있는 더미 계획("Ipsa deleniti aliqu") 정리 — 앱에 계획 삭제 기능이 없어 남아있음, 채점 기준상 문제는 아니지만 깔끔하지 않음

4. (선택) T07-C106 — Vercel 대시보드에서 Functions 로그를 직접 열어 비밀번호 원문이 안 찍히는지 최종 확인(AI는 접근 권한 없음)

아키텍처 결정 (카드 1 대응, 실제 채택안)

- ****직접 구현(bcrypt + JWT)****을 선택. Supabase Auth(GoTrue) 사용도 함께 검토했으나, "직접 만들어서 인증 흐름 자체를 증명하고 싶다"는 이유로 직접 구현 채택.
 - 이름·버전: `bcryptjs@2.4.3` (비밀번호 해시), `jsonwebtoken@9.0.2` (토큰 서명/검증), `cookie@0.6.0` (쿠키 직렬화) — 모두 `package.json` 에 고정.
 - 검토했지만 고르지 않은 방법: Supabase Auth(GoTrue) — 이미 Supabase 인프라를 쓰고 있어 붙이기는 더 쉬웠겠지만, 인증 로직을 라이브러리 뒤에 완전히 숨기게 되어 "어떻게 붙였는지 설명하기"라는 이 과제의 취지에는 직접 구현이 더 맞다고 판단.
 - 비밀번호 해시: `bcryptjs`, cost factor 12.
 - 세션 식별: **토큰(JWT)** — 서버가 발급한 JWT를 `httpOnly` 쿠키로 전달. 순수 `stateless` JWT는 로그아웃해도 만료 전까지 유효해 "로그아웃 후 거절" 요구를 못 지키므로, `sessions` 테이블을 두고 JWT의 `sid` 클레임으로 그 레코드를 가리켜 로그아웃 시 `revoked_at` 을 찍어 즉시 무효화한다.
 - 데이터 소유권: `plans / todos / execution_logs / plan_revisions` 에 `user_id` 컬럼 추가. RLS는 기존 `anon` 허용 정책을 전부 삭제해 잠그고, 백엔드(`/api`)만 `SUPABASE_SERVICE_ROLE_KEY` 로 RLS를 우회해 접근하며, 매 쿼리마다 애플리케이션 코드에서 `user_id = 로그인한 사용자` 를 직접 검사한다.

겪었던 문제 (기록용)

- **동적 catch-all 라우트 실패**: 처음엔 `api/data/[...route].js` 하나로 REST 스타일(`/api/data/plans/:id`) 라우팅을 짰는데, 배포 후 2단계 이상 경로가 Vercel 자체 404로 가로채지는 것을 발견(계획 수정 기능이 실제로 죽어 있었음). `id`를 쿼리스트링(`?id=`)으로 넘기는 정적 파일 라우팅으로 전면 재구성해 해결.
- **`req.query` 의 동적 세그먼트 미채움**: 위와 별개로 `req.query` 에 의존하지 않고 `req.url` 을 직접 파싱하도록 전 API에서 통일.